Signal Event

Signal Event

- Modeling a business process may require some kind of communication with other related business processes.
- The reason for this may be to wake up a process that was waiting for a particular situation to verify or to start a new process.
- The BPMN notation provides two different concepts for this purpose:
  - the Message and the Signal. Both of them are events.
- The main difference is that a Message is triggered to a single target while a Signal is broadcast to multiple targets.
- In other words, the Message is a point-to-point communication while the Signal is a broadcast communication.
- Signals are commonly used for general communication within and across Process Levels, across Pools and between business process diagrams.

Signal Event

- A Signal is composed of a name and a set of parameters.
- These parameters are sent when the signal is thrown; otherwise, they are received when the signal is caught.
- As the Signal is broadcast, different processes can have catching events listening for the same signal.
- Of course, there's also the chance of a process throwing a signal.
- Signal events are events which reference a named signal.
- A signal is an event of global scope (broadcast semantics) and is delivered to all active handlers.
- A Signal Event provides the facility of loosely coupling 'throwers' and 'catchers' by publish-subscribe integration.
- A 'thrower' will broadcast a signal rather than addressing it to a particular process; any process listening for that particular event could trigger a new instance using a Signal Start Event.

Signal Event

- A Signal can be thrown from either a throwing Intermediate Event or a throwing End Event, and can be caught in a Start Event or a catching Intermediate Event (including a boundary Signal Event).

Signal event types:

- Start Signal Event.
- Intermediate Throwing Signal Event.
- Intermediate Catching Event.
- Boundary Signal Event.

Signal Event

Start Signal Event

- A signal start event can be used to start a process instance using a named signal.
- When deploying a process definition with one or more signal start events, the following considerations apply:
  - The name of the signal start event must be unique across a given process definition, i.e., a process definition must not have multiple signal start events with the same name.
  - The engine throws an exception upon deployment of a process definition in case two or more signal start events reference the same signal or if two or more signal start events reference signals with the same signal name.

Signal Event

Start Signal Event

- When deploying a process definition with one or more signal start events, the following considerations apply:
  - Contrary to message start events, the name of the signal start event does not have to be unique across all deployed process definitions.
  - Process versioning: Upon deployment of a new version of a process definition, the signal subscriptions of the previous version are canceled. This is also the case for signal events that are not present in the new version.
  - A process instance of a process definition with one or more signal start events will be started, when a signal with the proper name is thrown.
  - The signal can either be thrown by a process instance (i.e., on intermediate throwing signal event or signal end event) or using the methods on the RuntimeService

Signal Event

Intermediate throw signal event:

- A signal intermediate throwing event throws a signal event for a defined signal.
- The signal is broadcast to all active handlers (i.e., all catching signal events).
- Signals can be published synchronously or asynchronously.
- In the default configuration, the signal is delivered synchronously.
- This means that the throwing process instance waits until the signal is delivered to all catching process instances.

Signal Event

Intermediate throw signal event:

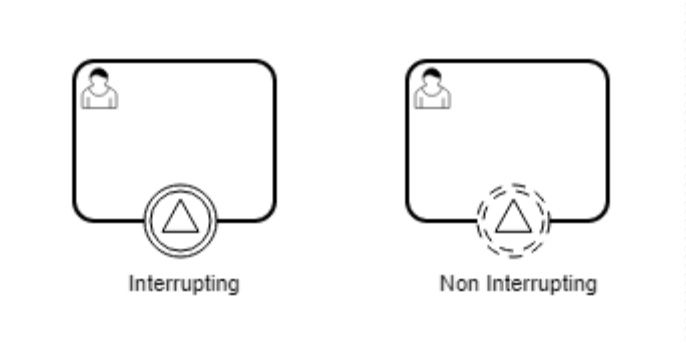
- The catching process instances are also notified in the same transaction as the throwing process instance, which means that if one of the notified instances produces a technical error (throws an exception), all involved instances fail.
- A signal can also be delivered asynchronously.
- In that case it is determined which handlers are active at the time the throwing signal event is reached.
- For each active handler, an asynchronous notification message (Job) is stored and delivered by the JobExecutor.

Signal Event

Intermediate catch signal event:

- The Intermediate Catching Signal Event is an event that waits for the associated Signal to be thrown.

Boundary signal event:



Signal Event

Boundary signal event:

- A boundary signal event listens to a signal being fired (from within the process instance or system-wide) while the activity upon which the event is defined is active.
- When an execution arrives in the activity to which the signal boundary event is attached, the signal boundary event catches signals with the proper name.
- Note: Contrary to other events, such as the error boundary event, a signal boundary event does not only catch signal events thrown from the scope it is attached to.
- A signal event has a global scope (broadcast semantics), meaning that the signal can be thrown from any place, even from a different process instance.